ARU8113

客户端集成指导手册 V1.0

Client Integration Guide V1.0

上海华依智造感知科技有限公司

文件修改履历

修订序号	修订内容	生效日期	版本	修订者	审核人
1	首次发布	2026-9-17	V1.0	韩立明	

目录

1 概述 4
1.1 产品与 SDK 角色 4
1.2 集成目标 4
1.3 本文档覆盖的对外接口 4
2 交付包 5
2.1 必须加入工程的文件 5
3 集成步骤 5
3.1 调用流程 5
4 平台移植（客户必须实现） 6
4.1 必须实现的函数符号 6
4.2 实现约束 7
4.3 SPI 约束 7
4.4 移植骨架（示意） 7
4.5 主控 MCU 资源约束 12
5 对外接口详细说明 12
5.1 hy_imu_get_version 12
5.2 hy_imu_init 13
5.3 hy_imu_proc_init 14
5.4 hy_imu_get_chip_id 15
5.5 hy_imu_set_factory_cal 16
5.6 hy_imu_get_factory_cal 17
5.7 hy_imu_read_sample_f32 17
5.8 hy_imu_calib_f32 18
5.9 hy_imu_proc_data_f32 19
5.10 hy_imu_proc_nav_f32 （可选） 20
6 完整集成示例 21

1 概述

1.1 产品与 SDK 角色

ARU8113 以 IMU 模组+软件 SDK 形式交付。SDK 负责完成 IMU 通信总线初始化协调、软复位、芯片识别、量程与 ODR 配置、传感器使能、启动期无效样本丢弃、工厂标定补偿以及一次姿态解算步进。客户端需完成芯片选通与时序、延时与时间基准、以及把工厂下发的标定表在初始化阶段写入 SDK。

1.2 集成目标

完成集成后，客户应用应能稳定执行如下闭环：

1. 链接静态库并实现全部 cust_imu_* 功能函数。
2. 初始化阶段调用 hy_imu_init() 和 hy_imu_proc_init() 完成 SDK 的初始化。
3. 初始化阶段按需调用读取芯片身份信息。
4. 初始化阶段调用 hy_imu_set_factory_cal() 完成设置 IMU 工厂内部的标定参数。
5. 运行时周期调用 hy_imu_read_sample_f32() -> hy_imu_calib_f32() -> hy_imu_proc_data_f32() 用于获取最终优化后的 IMU 数据。
6. 如果需要姿态数据，可在上一步用 hy_imu_proc_nav_f32() 替代 hy_imu_proc_data_f32() 函数，可获取欧拉角数据。

1.3 本文档覆盖的对外接口

接口	用途
hy_imu_get_version	读取已链接静态库的语义化版本
hy_imu_init	按实体上电、配置初始化 IMU 硬件
hy_imu_proc_init	按实体初始化算法
hy_imu_get_chip_id	读取连续芯片身份字符串，用于追溯 IMU
hy_imu_set_factory_cal	初始化阶段把工厂标定表反馈到 SDK 内部
hy_imu_get_factory_cal	从 SDK 回读内部标定参数
hy_imu_read_sample_f32	读一帧 float 类型的 IMU 原始数据
hy_imu_calib_f32()	读标定后 float 类型的 IMU 数据
hy_imu_proc_data_f32()	读取算法层处理后的 IMU 数据
hy_imu_proc_nav_f32	读取算法层处理后的 IMU 数据，并完成一次姿态解算

2 交付包

客户收到的发布包只包含可链接库、公开头文件、平台移植模板与本手册。详细目录如下：

```
hy_imu_sdk_release_<product>/
├── lib/
│   └── libhy_imu_<product>.a              /* 静态库 */
├── include/
│   ├── hy_imu_acq.h
│   ├── hy_imu_proc.h
│   ├── hy_imu_types.h
│   └── cust_imu_platform.h
├── port/
│   └── cust_imu_platform_template.c     /* 用于底层驱动集成示例文件 */
└── docs/
```

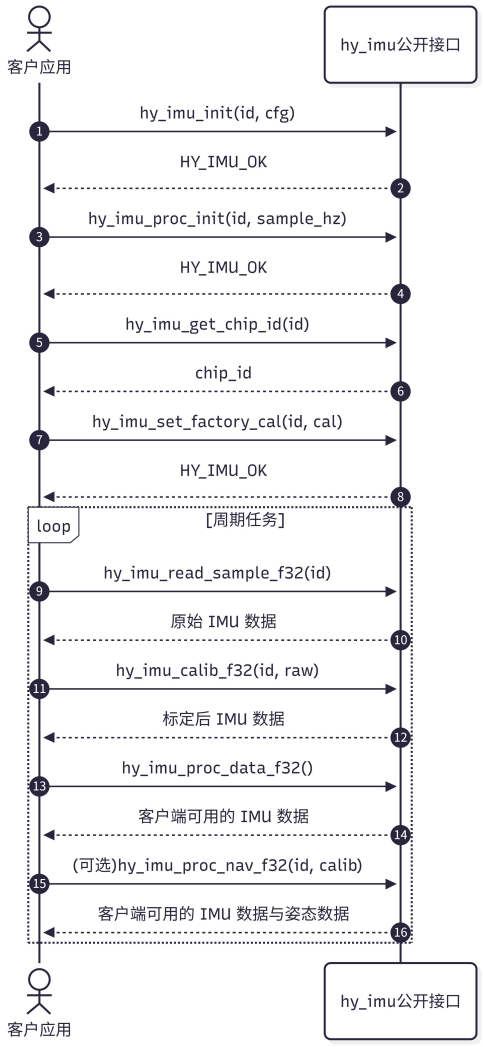
2.1 必须加入工程的文件

- 将 include/ 加入编译器头文件搜索路径。应用源文件包含 hy_imu_acq.h 和 hy_imu_proc.h 即可
- 将对应产品的 libhy_imu_*.a 加入链接。
- 将 port/cust_imu_platform_template.c 复制到客户工程并实现全部 cust_imu_*。该模板不得编进 SDK 静态库。

3 集成步骤

3.1 调用流程

客户应用与 SDK 的核心调用关系如下图，上半段为上电一次完成的动作，下半段为运行时周期调用的动作。



4 平台移植（客户必须实现）

cust_imu_* 是链接期必须提供的平台底层接口，SDK 通过这些函数访问主机外设。请以 port/cust_imu_platform_template.c 为起点，在客户工程中进行实现。

4.1 必须实现的函数符号

函数	职责
cust_imu_bus_init	按 serif 初始化 SPI4 与片选，配置时钟与引脚，若应用已先初始化外设，该实体函数可直接返回 HY_IMU_OK
cust_imu_read	从起始寄存器连续读 len 字节；len>0 时 buf 不得为空
cust_imu_write	向起始寄存器连续写 len 字节；len>0 时 buf 不得为空
cust_imu_delay_ms	毫秒级延时

函数	职责
cust_imu_get_host_time	返回当前的主机时间，用于采样时代入，SDK 不做处理，采样结束时随数据结果一同返回

4.2 实现约束

- SPI 超时必须返回错误码。
- SDK 传入的寄存器地址是逻辑地址，客户负责片选与 SPI 读地址位等主机侧约定。

4.3 SPI 约束

- SPI 模式为 Mode 0；时钟上限随 VDDIO 等级，建议时钟范围在 4M 到 12M 之间。
- 每次事务用片选包围；空闲时片选拉高。

4.4 移植骨架（示意）

```
#include "cust_imu_platform.h"

#include <stddef.h>
#include <string.h>

/** Invensense 4-wire SPI: set bit7 of the address byte to read. */
#define CUST_IMU_SPI_READ_BIT   (0x80U)

/** Address byte + payload. Keep large enough for the longest burst you use. */
#define CUST_IMU_SPI_MAX_XFER   (64U)

/**
 * @brief Select CS / controller for this entity (customer fills this in).
 * @param[in] bus  Non-NULL bus context (id and/or port_user).
 * @note Typical: GPIO CS from bus->id, or a cookie in bus->port_user.
 */
static void cust_imu_spi_cs_assert(const hy_imu_bus_ctx_t *bus)
{
    /* TODO: drive the CS for bus->id (or bus->port_user) low. */
    (void)bus;
}

/**
 * @brief Release CS after a SPI transaction (customer fills this in).
 * @param[in] bus  Non-NULL bus context.
 */
static void cust_imu_spi_cs_idle(const hy_imu_bus_ctx_t *bus)
{
    /* TODO: drive the CS for bus->id (or bus->port_user) high. */
    (void)bus;
}

/**
 * @brief Full-duplex SPI transfer of @p len bytes (customer fills this in).
 * @param[in]  bus  Non-NULL; select controller if several SPI units exist.
```

```
 * @param[in]  tx   Transmit buffer.
 * @param[out] rx   Receive buffer (may be ignored on write-only silicon).
 * @param[in]  len  Number of bytes (address + payload).
 * @return HY_IMU_OK or HY_IMU_ERR_BUS / HY_IMU_ERR_PARAM.
 */
static hy_imu_err_t cust_imu_spi_xfer(const hy_imu_bus_ctx_t *bus,
                                      const uint8_t *tx,
                                      uint8_t *rx,
                                      uint16_t len)
{
    hy_imu_err_t err = HY_IMU_OK;

    if ((bus == NULL) || (tx == NULL) || (rx == NULL) ||
        (len == 0U) || (len > CUST_IMU_SPI_MAX_XFER))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else
    {
        /* TODO: CS low -> SPI full-duplex of `len` bytes -> CS high.
         *       Map driver timeout / NACK to HY_IMU_ERR_BUS.
         *       Typical: SPI Mode 0, CS idle high. */
        (void)tx;
        (void)rx;
        err = HY_IMU_ERR_NOT_IMPL;
    }

    return err;
}

/**
 * @brief I2C register burst (customer fills this in).
 * @param[in]  bus   Non-NULL; use bus->i2c_addr (7-bit) and bus->port_user.
 * @param[in]  reg   Logical register pointer; do not OR 0x80.
 * @param[in]  tx    Write payload, or NULL on read.
 * @param[out] rx    Read payload, or NULL on write.
 * @param[in]  len   Payload length.
 * @param[in]  is_read  Non-zero for read.
 */
static hy_imu_err_t cust_imu_i2c_xfer(const hy_imu_bus_ctx_t *bus,
                                      uint8_t reg,
                                      const uint8_t *tx,
                                      uint8_t *rx,
                                      uint32_t len,
                                      uint8_t is_read)
{
    hy_imu_err_t err = HY_IMU_OK;

    if ((bus == NULL) || (bus->i2c_addr == 0U) || (bus->i2c_addr > 0x7FU))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if ((len > 0U) && (((is_read != 0U) && (rx == NULL)) ||
                            ((is_read == 0U) && (tx == NULL))))
    {
        err = HY_IMU_ERR_PARAM;
    }
```

```
    else
    {
        /* TODO: I2C write: START, addr, reg, tx[0..len-1], STOP.
         *       I2C read:  START, addr, reg, RESTART, addr|R, rx[0..len-1], STOP.
         *       Do not OR 0x80 into `reg`. */
        (void)reg;
        (void)tx;
        (void)rx;
        (void)len;
        err = HY_IMU_ERR_NOT_IMPL;
    }

    return err;
}

hy_imu_err_t cust_imu_bus_init(const hy_imu_bus_ctx_t *bus)
{
    hy_imu_err_t err = HY_IMU_OK;

    if (bus == NULL)
    {
        err = HY_IMU_ERR_PARAM;
    }
    else
    {
        switch (bus->serif)
        {
            case HY_IMU_SERIF_SPI4:
            case HY_IMU_SERIF_SPI3:
            {
                /*
                 * Called once per entity. Same SPI controller may already be up.
                 * Option A: clocks / SPI / pins already brought up by the
                 *           application; return HY_IMU_OK (idempotent).
                 * Option B: initialise the controller on first call; later
                 *           entities sharing it return HY_IMU_OK immediately.
                 */
                break;
            }
            case HY_IMU_SERIF_I2C:
            {
                /* Same as SPI: initialise the I2C unit at most once. */
                break;
            }
            default:
            {
                err = HY_IMU_ERR_PARAM;
                break;
            }
        }
    }

    return err;
}

hy_imu_err_t cust_imu_read(const hy_imu_bus_ctx_t *bus, uint8_t reg, uint8_t *buf, uint32_t
len)
```

```
{
    hy_imu_err_t err = HY_IMU_OK;
    uint8_t tx[CUST_IMU_SPI_MAX_XFER];
    uint8_t rx[CUST_IMU_SPI_MAX_XFER];
    uint16_t xfer_len;

    if (bus == NULL)
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if ((len > 0U) && (buf == NULL))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if (bus->serif == HY_IMU_SERIF_I2C)
    {
        err = cust_imu_i2c_xfer(bus, reg, NULL, buf, len, 1U);
    }
    else if ((bus->serif != HY_IMU_SERIF_SPI4) && (bus->serif != HY_IMU_SERIF_SPI3))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if ((len + 1U) > CUST_IMU_SPI_MAX_XFER)
    {
        err = HY_IMU_ERR_PARAM;
    }
    else
    {
        xfer_len = (uint16_t)(len + 1U);
        (void)memset(tx, 0, xfer_len);
        (void)memset(rx, 0, xfer_len);

        /* Logical address from the SDK. 4-wire SPI must OR the read bit. */
        tx[0] = (uint8_t)(reg | CUST_IMU_SPI_READ_BIT);

        cust_imu_spi_cs_assert(bus);
        err = cust_imu_spi_xfer(bus, tx, rx, xfer_len);
        cust_imu_spi_cs_idle(bus);
        if ((err == HY_IMU_OK) && (len > 0U))
        {
            /* Skip the dummy byte clocked out while the address is sent. */
            (void)memcpy(buf, &rx[1], len);
        }
    }

    return err;
}

hy_imu_err_t cust_imu_write(const hy_imu_bus_ctx_t *bus, uint8_t reg, const uint8_t *buf,
uint32_t len)
{
    hy_imu_err_t err = HY_IMU_OK;
    uint8_t tx[CUST_IMU_SPI_MAX_XFER];
    uint8_t rx[CUST_IMU_SPI_MAX_XFER];
    uint16_t xfer_len;

    if (bus == NULL)
```

```
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if ((len > 0U) && (buf == NULL))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if (bus->serif == HY_IMU_SERIF_I2C)
    {
        err = cust_imu_i2c_xfer(bus, reg, buf, NULL, len, 0U);
    }
    else if ((bus->serif != HY_IMU_SERIF_SPI4) && (bus->serif != HY_IMU_SERIF_SPI3))
    {
        err = HY_IMU_ERR_PARAM;
    }
    else if ((len + 1U) > CUST_IMU_SPI_MAX_XFER)
    {
        err = HY_IMU_ERR_PARAM;
    }
    else
    {
        xfer_len = (uint16_t)(len + 1U);
        (void)memset(tx, 0, xfer_len);
        (void)memset(rx, 0, xfer_len);

        /* Logical address as-is. Do NOT set bit7 (that would turn this into a read). */
        tx[0] = reg;
        if (len > 0U)
        {
            (void)memcpy(&tx[1], buf, len);
        }

        cust_imu_spi_cs_assert(bus);
        err = cust_imu_spi_xfer(bus, tx, rx, xfer_len);
        cust_imu_spi_cs_idle(bus);
    }

    return err;
}

void cust_imu_delay_ms(uint32_t ms)
{
    if (ms > 0U)
    {
        /* TODO: RTOS / OS tick delay for `ms` milliseconds.
         *       Example: OSIF_TimeDelay(ms); */
        (void)ms;
    }
}

uint64_t cust_imu_get_host_time(void)
{
    /* TODO: return the host clock used to stamp hy_imu_read_sample_f32().
     *       Unit and epoch are application-defined; the SDK forwards the
     *       value without conversion. */
    return 0U;
}
```

4.5 主控 MCU 资源约束

本 SDK 以静态库链接到客户主控，无动态内存分配。

项目	建议值	说明
主频	200 MHz	建议值，我方自测平台主频约 80 MHz、优化等级 O1，单路 IMU 完整跑一遍 SDK 数据通路（读样 → 标定 → 算法）大约 700 µs 量级
Flash	25KB	几乎不随 IMU 实体数增加而增加
RAM	4KB	每增加一个实体 RAM 额外占用 3.58KB
任务栈	1KB	任务栈深度，不随 IMU 实体数增加而增加

5 对外接口详细说明

5.1 hy_imu_get_version

读取已链接静态库的语义化版本。这是 SDK 库版本。可在 hy_imu_init() 之前调用。

原型

```
hy_imu_err_t hy_imu_get_version(hy_imu_version_t *version);
```

参数

参数	方向	类型	说明
version	Out	hy_imu_version_t *	成功时写入 major / minor / patch

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	version 为空

版本结构

```
typedef struct {
    uint8_t major;
    uint8_t minor;
    uint8_t patch;
} hy_imu_version_t;
```

示例

```
hy_imu_version_t ver;
if (hy_imu_get_version(&ver) == HY_IMU_OK) {
```

```
    /* ver.major, ver.minor, ver.patch */
}
```

5.2 hy_imu_init

按实体初始化配置 IMU 硬件，并丢弃启动阶段不稳定样本。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别初始化。

原型

```
hy_imu_err_t hy_imu_init(hy_imu_id_t id, const hy_imu_config_t *cfg);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 . . 11 编库槽位数-1, 最大支持 12 个实体
cfg	In	const hy_imu_config_t *	调用期间只读，是 IMU 配置参数

返回值

返回值	含义
HY_IMU_OK	成功，此后可调用 chip id / 标定 / 读导航
HY_IMU_ERR_PARAM	cfg 为空或字段非法；模块保持未初始化
HY_IMU_ERR_STATE	已经初始化，禁止重复 init
HY_IMU_ERR_IO	平台总线初始化失败（检查 cust_imu_bus_init）
HY_IMU_ERR_BUS	寄存器读写失败
HY_IMU_ERR_WHOAMI	芯片识别失败
HY_IMU_ERR_TIMEOUT	复位完成等待超时

调用约束

- 调用前必须已实现并链接全部 cust_imu_*。

hy_imu_config_t 定义

```
typedef struct
{
    hy_imu_serif_t       serif;             /**< Host interface type */
    uint8_t            i2c_addr;  /**< 7-bit I2C address, per entity; ignored for SPI */
    void                *port_user;         /**< Optional cookie forwarded to the port */
    hy_imu_accel_fsr_t   accel_fsr;         /**< Accel full-scale range */
    hy_imu_gyro_fsr_t    gyro_fsr;          /**< Gyro full-scale range */
    hy_imu_odr_t         accel_odr;         /**< Accel ODR */
    hy_imu_odr_t         gyro_odr;          /**< Gyro ODR */
    hy_imu_sensor_mode_t accel_mode;     /**< Accel mode enabled during hy_imu_init() */
    hy_imu_sensor_mode_t gyro_mode;       /**< Gyro mode enabled during hy_imu_init() */
```

```
    hy_imu_ln_bw_t       accel_ln_bw;       /**< Accel LN LPF bandwidth */
    hy_imu_ln_bw_t       gyro_ln_bw;        /**< Gyro LN LPF bandwidth */
    hy_imu_lp_avg_t      accel_lp_avg;      /**< Accel LP average */
    hy_imu_lp_avg_t      gyro_lp_avg;       /**< Gyro LP average */
    hy_imu_sample_mode_t sample_mode;       /**< Polling or interrupt-driven sampling */
    hy_imu_int_config_t  int_cfg;         /**< INT pin config (used in interrupt mode) */
    uint16_t             discard_samples;   /**< Samples to discard after start (startup
settle) */
} hy_imu_config_t;
```

示例

```
hy_imu_config_t cfg = HY_IMU_CONFIG_DEFAULT;
cfg.serif     = HY_IMU_SERIF_SPI4;
cfg.accel_odr = HY_IMU_ODR_1600_HZ;
cfg.gyro_odr  = HY_IMU_ODR_1600_HZ;
cfg.sample_mode = HY_IMU_SAMPLE_MODE_POLLING;

hy_imu_err_t err = hy_imu_init(0U, &cfg);
if (err != HY_IMU_OK) {
    /* 记录 err */
}
```

5.3 hy_imu_proc_init

初始化一个 IMU 实体的算法状态，不访问总线。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别初始化。

原型

```
hy_imu_err_t hy_imu_proc_init(hy_imu_id_t id, float sample_hz);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1, 最大支持 12 个实体
sample_hz	In	float	后续 calib/nav 的调用频率

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	id 越界，或 sample_hz 非有限 / ≤0
HY_IMU_ERR_STATE	该实体已经初始化

示例

```
hy_imu_err_t err = hy_imu_proc_init(0U, 1000);
if (err != HY_IMU_OK) {
    /* 记录 err，禁止后续 calib/nav 操作 */
}
```

5.4 hy_imu_get_chip_id

格式化为连续可打印字符串，用于模组追溯与产线对账。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_get_chip_id(hy_imu_id_t id, hy_imu_chip_id_t *chip_id);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1, 最大支持 12 个实体
chip_id	Out	hy_imu_chip_id_t *	返回 chip id 信息

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	chip_id 为空或 id 越界
HY_IMU_ERR_STATE	尚未 hy_imu_init 成功
HY_IMU_ERR_BUS	总线或 OTP 访问失败
HY_IMU_ERR_TIMEOUT	有界等待超时
HY_IMU_ERR_UNSUPPORTED	当前链接的产品不支持该读取
HY_IMU_ERR_INTERNAL	解码或内部不变量失败

身份结构

```
typedef struct {
    char str[16];         /* 15 字符 + NUL，例如 "E9G6S1910011425" */
} hy_imu_chip_id_t;
```

- id 以 NUL 结尾，有效长度为 15，缓冲区为 16 字节。
- 仅初始化阶段调用。调用期间库会短暂切换芯片时钟并打开 OTP，返回前恢复。

示例

```
hy_imu_chip_id_t chip;
if (hy_imu_get_chip_id(0U, &chip) == HY_IMU_OK) {
    /* chip.str : 例如 "E9G6S1910011425" */
```

```
}
```

5.5 hy_imu_set_factory_cal

工厂完成单板标定后，把标定表交给客户保存（例如 MCU Flash）。客户在软件初始化阶段主动调用本接口，把标定参数交给 SDK。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_set_factory_cal(
    hy_imu_id_t id,
    const hy_imu_factory_cal_t *cal);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1, 最大支持 12 个实体
cal	In	const hy_imu_factory_cal_t *	客户填入的标定表

返回值

返回值	含义
HY_IMU_OK	调用成功
HY_IMU_ERR_PARAM	id 越界或 cal 为空
HY_IMU_ERR_STATE	该实体尚未 hy_imu_init

调用时机

1. hy_imu_init() 已返回 HY_IMU_OK。
2. 客户从非易失存储读出本板标定表，并完成自身 CRC/版本等校验。

标定表 hy_imu_factory_cal_t

布局与工厂输出一致。客户按相同内存布局存储与拷贝即可，不必在应用中实现补偿公式。

```
typedef struct {
    float gyro_temp[3][4];
    float acce_temp[3][4];
    float acce_cal[3][4];
    float gyro_cal[3][4];
} hy_imu_factory_cal_t;
```

示例

```
hy_imu_factory_cal_t cal;
```

```
// todo file cal
(void)hy_imu_set_factory_cal(0U, &cal);
```

5.6 hy_imu_get_factory_cal

获取 SDK 内部标定参数。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_get_factory_cal(
    hy_imu_id_t id,
    hy_imu_factory_cal_t *cal);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1, 最大支持 12 个实体
cal	Out	hy_imu_factory_cal_t *	返回 SDK 内部标定参数，用于设置后回读验证

返回值

返回值	含义
HY_IMU_OK	调用成功
HY_IMU_ERR_PARAM	id 越界或 cal 为空
HY_IMU_ERR_STATE	该实体尚未 hy_imu_init

示例

```
hy_imu_factory_cal_t cal;

(void)hy_imu_get_factory_cal(0U, &cal);
```

5.7 hy_imu_read_sample_f32

读取浮点类型的 IMU 原始数据。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_read_sample_f32(hy_imu_id_t id, hy_imu_sample_f32_t *out);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1, 最大支持 12 个实体
out	Out	hy_imu_sample_f32_t s*	IMU 原始数据

hy_imu_sample_f32_t 定义

```
typedef struct
{
    float    accel_x_g;     /**< Accel X in g, product frame */
    float    accel_y_g;     /**< Accel Y in g, product frame */
    float    accel_z_g;     /**< Accel Z in g, product frame */
    float    gyro_x_dps;    /**< Gyro X in degrees per second, product frame */
    float    gyro_y_dps;    /**< Gyro Y in degrees per second, product frame */
    float    gyro_z_dps;    /**< Gyro Z in degrees per second, product frame */
    float    temp_c;        /**< Temperature in degrees Celsius */
    uint16_t timestamp;     /**< not available */
    uint64_t host_time;     /**< Host clock from cust_imu_get_host_time(); opaque */
} hy_imu_sample_f32_t;
```

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	id 越界或 out 为空
HY_IMU_ERR_STATE	该实体未处于 RUNNING
HY_IMU_ERR_BUS	总线失败

示例

```
hy_imu_sample_f32_t sample;
hy_imu_err_t        err;

err = hy_imu_read_sample_f32(0U, &sample);
if (err == HY_IMU_OK)
{
    /* sample.host_time：本次采集打上的客户时钟
     * sample.accel_*_g / gyro_*_dps / temp_c：原始物理量
     * 若要标定或姿态，把 sample 交给 hy_imu_calib_f32 / hy_imu_proc_nav_f32
     */
}
```

5.8 hy_imu_calib_f32

读取标定后的加速度、角速度，以及温度。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_calib_f32(hy_imu_id_t id,
                              const hy_imu_sample_f32_t *in,
                              hy_imu_sample_f32_t *out);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1,最大支持 12 个实体
in	In	hy_imu_sample_f32_t*	IMU 原始数据，通过 hy_imu_read_sample_f32 函数获取
out	Out	hy_imu_sample_f32_t s*	IMU 标定后的数据

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	id 越界或 out 为空
HY_IMU_ERR_STATE	该实体未初始化

示例

```
hy_imu_sample_f32_t raw;
hy_imu_sample_f32_t calib;

hy_imu_read_sample_f32(0U, &raw);
hy_imu_calib_f32(0U, &raw, &calib);  /* calib 为标定后的数据 */
```

5.9 hy_imu_proc_data_f32

读取算法优化后的 IMU 数据。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_proc_data_f32(hy_imu_id_t id,
                                  const hy_imu_sample_f32_t *in,
                                  hy_imu_sample_f32_t *out);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 .. 11 编库槽位数-1,最大支持 12 个实体
in	In	hy_imu_sample_f32_t*	IMU 标定后的数据，通过 hy_imu_calib_f32 函数获取
out	Out	hy_imu_sample_f32_t s*	算法优化后的 IMU 数据

返回值

返回值	含义
HY_IMU_OK	成功
HY_IMU_ERR_PARAM	id 越界或 out 为空
HY_IMU_ERR_STATE	该实体未初始化

示例

```
hy_imu_sample_f32_t raw;
hy_imu_sample_f32_t calib;
hy_imu_sample_f32_t proc;

hy_imu_read_sample_f32(0U, &raw);
hy_imu_calib_f32(0U, &raw, &calib);  /* calib 为标定后的数据 */
hy_imu_proc_data_f32(0U, &calib, &proc); /* proc 为算法优化后的数据，最后客户使用的*/
```

5.10 hy_imu_proc_nav_f32 （可选）

如果需要姿态信息，可用该接口替换 hy_imu_proc_data_f32 函数接口。该接口返回算法处理后的 IMU 数据以及姿态解算信息。一个 id 对应一个 IMU 实体，如果多个 IMU 需分别调用该接口。

原型

```
hy_imu_err_t hy_imu_proc_nav_f32(hy_imu_id_t id,
                                 const hy_imu_sample_f32_t *in,
                                 hy_imu_nav_f32_t *out);
```

参数

参数	方向	类型	说明
id	In	hy_imu_id_t	0 ..11 编库槽位数-1,最大支持 12 个实体
In	In	hy_imu_sample_f32_t	IMU 标定后的数据，通过 hy_imu_calib_f32 函数获取
out	Out	hy_imu_nav_f32_t *	成功时返回最新一帧导航数据

返回值

返回值	含义
HY_IMU_OK	成功，out 为新快照
HY_IMU_ERR_PARAM	id 越界或 out 为空
HY_IMU_ERR_STATE	未处于可采样状态（hy_imu_proc_init 未成功）

hy_imu_nav_f32_t 定义

```
typedef struct
{
    float accel_x_g;
    float accel_y_g;
    float accel_z_g;
    float gyro_x_dps;
    float gyro_y_dps;
    float gyro_z_dps;
    float temp_c;
    float quat_w;
    float quat_x;
    float quat_y;
    float quat_z;
    float roll_rad;
    float pitch_rad;
    float yaw_rad;
    uint8_t ahrs_init_state; /*< Internal AhrsInitStateType */
    double host_time;        /*< Copied from the input sample; opaque host clock */
} hy_imu_nav_f32_t;
```

示例

```
hy_imu_sample_f32_t raw;
hy_imu_sample_f32_t calib;
hy_imu_nav_f32_t nav;

hy_imu_read_sample_f32(0U, &raw);
hy_imu_calib_f32(0U, &raw, &calib);  /* calib 为标定后的数据 */
hy_imu_proc_nav_f32(0U, &calib, &nav); /* nav 为算法优化后的 IMU 数据以及姿态解算数据 */
```

6 完整集成示例

下列示例给出最小闭环：版本查询、配置与 init、芯片身份、工厂标定下发、导航读取。

```
#include "hy_imu_acq.h"
#include "hy_imu_proc.h"
#include "cust_imu_platform.h"

#include <stdint.h>
#include <stdio.h>

#define DEMO_IMU_COUNT  (5U)

static const hy_imu_id_t s_ids[DEMO_IMU_COUNT] = {
    (hy_imu_id_t)0U,
    (hy_imu_id_t)1U,
    (hy_imu_id_t)2U,
    (hy_imu_id_t)3U,
    (hy_imu_id_t)4U
};

static uint32_t s_sample_count = 0U;

static void demo_build_spi_config(hy_imu_config_t *cfg)
```

```
{
    const hy_imu_config_t defaults = HY_IMU_CONFIG_DEFAULT;

    *cfg = defaults;
    cfg->serif       = HY_IMU_SERIF_SPI4;
    cfg->port_user   = NULL;
    cfg->accel_fsr   = HY_IMU_ACCEL_FSR_8G;
    cfg->gyro_fsr    = HY_IMU_GYRO_FSR_2000_DPS;
    cfg->accel_odr   = HY_IMU_ODR_1600_HZ;
    cfg->gyro_odr    = HY_IMU_ODR_1600_HZ;
    cfg->accel_mode  = HY_IMU_SENSOR_MODE_LN;
    cfg->gyro_mode   = HY_IMU_SENSOR_MODE_LN;
    cfg->accel_ln_bw = HY_IMU_LN_BW_ODR_DIV_4;
    cfg->gyro_ln_bw  = HY_IMU_LN_BW_ODR_DIV_4;
    cfg->sample_mode = HY_IMU_SAMPLE_MODE_POLLING;
}

static void demo_print_nav(hy_imu_id_t id, uint32_t index, const hy_imu_nav_f32_t *nav)
{
    (void)printf(
        "[id%u %lu] acc %.3f %.3f %.3f g | gyr %.2f %.2f %.2f dps | "
        "quat %.3f %.3f %.3f %.3f | rpy %.3f %.3f %.3f rad\r\n",
        (unsigned int)id,
        (unsigned long)index,
        (double)nav->accel_x_g,
        (double)nav->accel_y_g,
        (double)nav->accel_z_g,
        (double)nav->gyro_x_dps,
        (double)nav->gyro_y_dps,
        (double)nav->gyro_z_dps,
        (double)nav->quat_w,
        (double)nav->quat_x,
        (double)nav->quat_y,
        (double)nav->quat_z,
        (double)nav->roll_rad,
        (double)nav->pitch_rad,
        (double)nav->yaw_rad);
}

static void demo_on_1ms_task(void)
{
    hy_imu_sample_f32_t samples[DEMO_IMU_COUNT];
    hy_imu_sample_f32_t calib;
    hy_imu_nav_f32_t    nav;
    hy_imu_err_t        err;
    unsigned int        i;

    /* Phase 1: read raw samples from all IMUs first (minimise bus latency skew). */
    for (i = 0U; i < DEMO_IMU_COUNT; i++)
    {
        hy_imu_read_sample_f32(s_ids[i], &samples[i]);
    }

    /* Phase 2: calib + nav for each successfully read sample. */
    for (i = 0U; i < DEMO_IMU_COUNT; i++)
    {
        hy_imu_calib_f32(s_ids[i], &samples[i], &calib);
```

```
        err = hy_imu_proc_nav_f32(s_ids[i], &calib, &nav); /* 如果不需要姿态数据，可用 hy_imu_proc_data_f32 函数替换 */

        if (i == 0U)
        {
            s_sample_count++;
        }
        demo_print_nav(s_ids[i], s_sample_count, &nav);
    }
}

static void demo_print_identity(hy_imu_id_t id)
{
    hy_imu_chip_id_t chip_id = {{0}};

    if (hy_imu_get_chip_id(id, &chip_id) == HY_IMU_OK)
    {
        (void)printf("imu %u chip sn %s\r\n", (unsigned int)id, chip_id.str);
    }
}

int main(void)
{
    hy_imu_err_t    err = HY_IMU_OK;
    hy_imu_config_t cfg;
    unsigned int    i;
    hy_imu_factory_cal_t cal;

    (void)printf("HY IMU SDK demo, acq %s, %u IMUs\r\n",
                 hy_imu_get_version_string(),
                 (unsigned int)DEMO_IMU_COUNT);

    demo_build_spi_config(&cfg);

    for (i = 0U; i < DEMO_IMU_COUNT; i++)
    {
        err = hy_imu_init(s_ids[i], &cfg);
        if (err != HY_IMU_OK)
        {
            (void)printf("hy_imu_init(%u) failed: %ld\r\n",
                         (unsigned int)s_ids[i], (long)err);
            return (int)err;
        }
    }

    for (i = 0U; i < DEMO_IMU_COUNT; i++)
    {
        err = hy_imu_proc_init(s_ids[i], 1000.0f);
        if (err != HY_IMU_OK)
        {
            (void)printf("hy_imu_proc_init(%u) failed: %ld\r\n",
                         (unsigned int)s_ids[i], (long)err);
            return (int)err;
        }
    }
```

```
    for (i = 0U; i < DEMO_IMU_COUNT; i++)
    {
        demo_print_identity(s_ids[i]);

         /* todo : fill cal */;
         hy_imu_set_factory_cal(s_ids[i], &cal);
    }

    for (;;)
    {
        demo_on_1ms_task();
    }
}
```